

포인터 · 배열 · 문자열 · 동적 메모리

주소를 다루는 언어. 어제 세 번 미뤘던 `&v`, `const char *`, `&s` 를 오늘 전부 회수합니다



오늘의 8교시

오전은 주소·배열, 오후는 문자열·힙 · 오늘 함정은 대부분 세그폴트로 끝납니다 — 정상입니다

1

포인터 기초

&, *, swap, 쓰레기 포인터

2

포인터 산술·const

p+1 보폭, const 읽기

3

배열과 포인터

감쇠, sizeof 함정, 2차원

4

정렬과 검색

버블, 이진, off-by-one

5

문자열

%0, char[] vs char*, 직접 구현

6

안전한 처리·파싱

fgets, snprintf, strtok, strtol

7

동적 메모리

malloc/realloc/free, valgrind

8

종합 실습

센서 로그 분석기

준비 — 오늘은 -g 를 붙인다

세그폴트가 났을 때 몇 번째 줄인지 알려면 디버그 정보가 필요합니다

```
$ gcc -Wall -Wextra -g -o app app.c && ./app

# 세그폴트가 나면 이 한 줄로 위치를 찾는다
$ gdb -batch -ex run -ex bt ./app 2>&1 | tail -5
#0  main () at wild.c:5      ← 죽은 줄 번호

# 7교시부터
$ valgrind --leak-check=full ./app
```

오늘 나오는 세그폴트는 전부 의도된 것입니다. 죽는 줄 번호를 gdb 로 찾는 연습 자체가 목표.

어제 미뤄둔 세 가지

`(unsigned char *)&v`

1·2교시

`const char *level()`

2·5교시

`judge(..., &s, &b)`

1교시 첫 20분

어제의 Reading 구조체와 센서 프레임
\$T=25.3,H=60,D=120* 을 오늘 문자열로 파싱하고
힙에 쌓습니다.

주소는 그냥 숫자다 — & 와 *

1교시

```
int x = 5;
int *p = &x;           // p 에 x 의 주소

printf("%d\n", x);      // 5
printf("%p\n", &x);     // 0x7ffc...20
printf("%p\n", p);      // 같은 값
printf("%d\n", *p);     // 5 (화살표 따라가 읽기)



*p = 42;           // 화살표 따라가 쓰기


printf("%d\n", x);      // 42
```



<code>int *p</code>	"p 를 역참조하면 int" — 오른쪽에서 왼쪽으로
<code>sizeof(p)</code>	8 — 주소 폭 (어제 <code>sizeof(int *)</code> 의 답)
<code>sizeof(*p)</code>	4 — 가리키는 것의 크기

어제 `inc(int x){ x++; }` 가 실패한 이유: 값을 복사했으니 화살표가 없었다. 오늘: 주소를 넘겨 화살표를 만든다.

📺 D2-1 포인터 화살표 뷰어 — `p = &x, *p = 42`

swap 을 성공시키기 · ⚠ 함정 둘

1교시

```
void inc_wrong(int x) { x++; } // Day 1: 복사본
void inc_right(int *p) { (*p)++; } // 주소 → 원본

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

int x = 5, y = 9;
inc_wrong(x); // x = 5
inc_right(&x); // x = 6
swap(&x, &y); // x = 9, y = 6
```

어제 judge(ans, g, &s, &b) 가 두 값을 돌려줄 수 있었던 이유가 inc_right 입니다.

⚠ 어디도 가리키지 않는 포인터

```
int *p; // 쓰레기 주소
*p = 10; // Segmentation fault
$ gdb -batch -ex run -ex bt ./wild | tail -3
#0 main () at wild.c:5
```

⚠ *p++ 와 (*p)++

```
int arr[3] = {3, 1, 2}; int *p = arr;
printf("%d ", *p++); // 3, p → arr[1]
printf("%d ", *p); // 1
(*p)++; // arr[1] = 2
```

후위 ++ 가 * 보다 우선 → *p++ 는 *(p++). 어제 정답의 (*strike)++ 괄호가 이것.

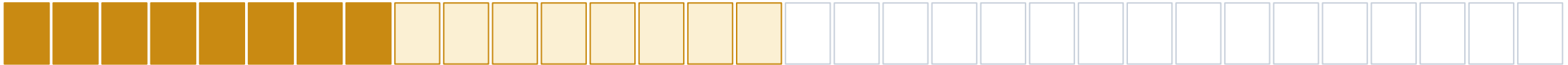
p + 1 은 1바이트가 아니다

2교시

int *p p + 1 (4바이트 뒤)



double *q q + 1 (8바이트 뒤)



```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;
p          // ...90
p + 1      // ...94   ← 4씩
p + 2      // ...98

*(p + 2)           // 30  == arr[2]
&arr[4] - &arr[0]  // 4   (요소 개수, 바이트 아님)
```

```
// 센서 4바이트 → 거리 300, 습도 60 (LE)
uint8_t buf[4] = {0x2C, 0x01, 0x3C, 0x00};

// ① 캐스팅: 정렬 맞을 때만
uint16_t *w = (uint16_t *)buf;   // w[0]=300 w[1]=60

// ② 바이트 조립: 어디서나 (임베디드 권장)
uint16_t d = buf[0] | (buf[1] << 8);  // 300
```

const 는 오른쪽에서 왼쪽으로 읽는다

2교시

선언

읽는 법

값 바꾸기

포인터 옮기기

`const int *p1`

p1 은 const int 를 가리킨다

`*p1 = 9` X

`p1 = &b` ✓

`int *const p2`

p2 는 int 를 가리키는 const

`*p2 = 9` ✓

`p2 = &b` X

`const int *const p3`

둘 다 const

`*p3 = 9` X

`p3 = &b` X

```
int sum(const int *a, int n);           // "읽기만 한다" 약속
int is_valid(const int g[3]);           // 어제 숫자 야구

size_t my_strlen(const char *s);        // 5교시
int parse_frame(const char *line, Reading *out); // 6교시: line 은 읽기만, out 은 쓴다
```

함수 안에서 실수로 `a[i] = ...` 를 쓰면 컴파일 에러. 호출자는 배열이 훼손되지 않을 것을 안다. 인자 타입만 봐도 함수의 의도가 읽힌다.

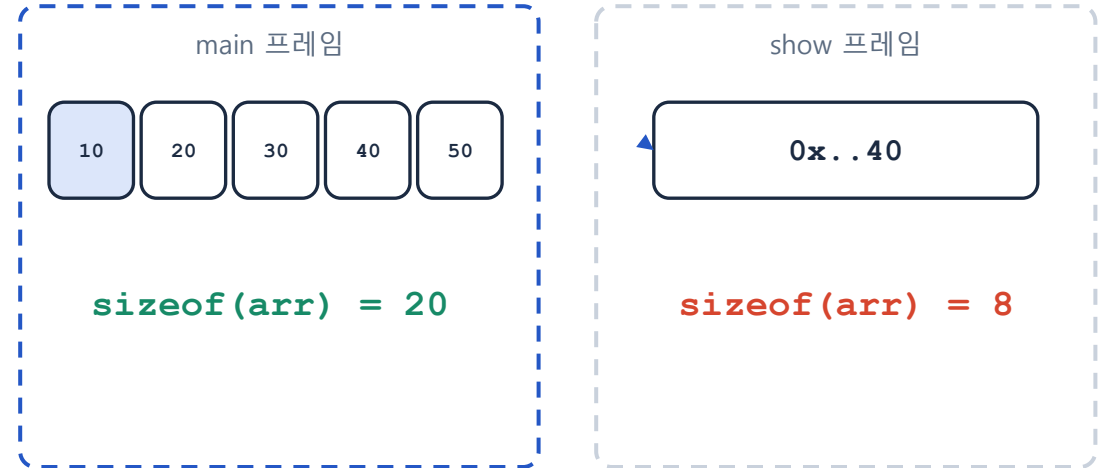
배열은 함수에 넘기는 순간 포인터가 된다

3교시

```
int arr[5] = {10, 20, 30, 40, 50};
arr[2] == *(arr + 2) == 2[arr]    // 30
sizeof(arr)                        // 20
sizeof(arr) / sizeof(arr[0])      // 5 ← 요소 개수 관용구

void show(int arr[]) {             // 사실은 int *arr
    sizeof(arr)                    // 8 !!
}

int sum(const int *a, int n) // 그래서 길이를 따로
sum(arr, 5);
```



첫 요소의 주소 하나만 넘어간다 (감쇠, decay). 요소가 5개라는 정보는 사라진다.

C 라이브러리가 `fwrite(buf, size, n, fp)`, `qsort(arr, n, size, cmp)` 처럼 (배열, 길이) 를 짝으로 받는 이유.

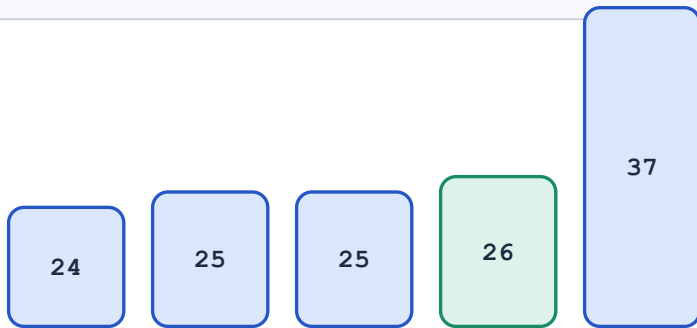
2차원 `int m[2][3]` 은 한 덩어리 (1 2 3 4 5 6). `m[1][0]` 은 12바이트 뒤. 함수에 넘길 땐 `int m[][3]` — 열 수가 있어야 주소 계산 가능.

정렬과 검색 — swap 을 써먹는 첫 자리

4교시

```
int bubble(int *a, int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)    // 뒤 i개 확정
            if (a[j] > a[j + 1])
                swap(&a[j], &a[j + 1]);
}
// {26,25,37,25,24} → 24 25 25 26 37 (7 swaps)
```

```
int bsearch_int(const int *a, int n, int key) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // 오버플로 회피
        if (a[mid] == key) return mid;
        if (a[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
    return -1;    // 인덱스는 0 이상이라 겹치지 않음
}
```



key=26: mid=2(25) < 26 → lo=3 → mid=3 → 찾음

△ $j < n - i$ 로 쓰면 $a[j+1]$ 이 $a[n]$ — 배열 밖. 컴파일러는 막지 않는다
이진 검색은 정렬 전제. 정렬 안 된 배열이면 버린 쪽에 답이 있을 수 있다.

구조체 정렬(8교시): Reading $t = a[j]$; $a[j] = a[j+1]$; $a[j+1] = t$; — 구조체는 = 로 통째로 복사된다. 데이터가 많으면 Day 3 qsort.

⚠ 함정 — 배열 밖은 조용히 남의 것을 덮는다

4교시

```
struct Dev { char name[8]; int level; };
```

```
struct Dev *d = malloc(sizeof *d);  
strcpy(d->name, "ESP"); d->level = 1;  
// before: name=ESP level=1
```

```
strcpy(d->name, "ESP32-SENSOR"); // 13글자 → 8칸  
// after : name=ESP32-SENSOR  
// level=1380930382 (0x524F534E)
```



name[0..7]

level (4바이트)

0x524F534E = 'N' 'S' 'O' 'R' (리틀 엔디안)

```
warning: '__builtin_memcpy' writing 13 bytes  
into a region of size 8 overflows the destination  
*** stack smashing detected *** ← 스택이면 glibc 가 잡  
기도
```

경고는 나지만 실행은 된다. 스택이면 운 좋게 잡히고, 힙이면 조용히 옆 필드가 바뀐다. MCU 에는 보호가 없다. 입력이 사용자에게서 오면 이것이 buffer overflow 취약점.

"ABC" 는 4바이트 — char[] vs char*

5교시

```
char s1[] = "ABC";    // 스택에 복사 — 수정 가능
char *s2 = "ABC";    // 읽기 전용 리터럴을 가리킴

sizeof(s1) // 4      (A B C \0)
sizeof(s2) // 8      (포인터)
strlen(s1) // 3      (\0 전까지)

s1[0] = 'X';    // XBC
s2[0] = 'X';    // Segmentation fault
const char *s2 = "ABC"; // → 컴파일 에러로 바꾸기
```

스택



s1[] (4B)

읽기 전용 영역 (.rodata)



s2 (8B)



sizeof · strlen · 포인터 크기 — 세 숫자를 구분하는 것이 오늘의 절반.

```
size_t my_strlen(const char *s) {
    const char *p = s;
    while (*p) p++;           // '\0' (값 0) 까지
    return p - s;             // 포인터 뺄셈 = 글자 수
}
while ((*d++ = *s++) != '\0') {} // strcpy 한 줄
```

프레임에서 값 찾기 — strstr, strchr, atof

5교시

```
const char *frame = "$T=25.3,H=60,D=120*";
const char *t = strstr(frame, "T=");    // "T=" 시작 위치
const char *e = strchr(frame, '*');      // '*' 위치

printf("%s\n", t + 2);                  // 25.3,H=60,D=120* (끝까지)
printf("%.1f\n", atof(t + 2));          // 25.3 (숫자 아닌 곳에서 멈춤)
printf("%ld\n", e - frame);              // 18 ('*' 의 인덱스)
```

```
int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned
char)*b;
}

strcmp(a, b) == 0    → 같다
if (strcmp(a, b))    → "다르면" ← 자주 틀림
```

반환 0 = 같음, 음수 = a 가 앞, 양수 = a 가 뒤.

atoi/atof 는 실패해도 0 을 돌려준다 → "0" 과 구분 불가. 6교시에서 strtol 의 끝 포인터로 바꾼다.

안전하게 읽고, 안전하게 자른다

6교시

fgets — 버퍼 크기를 아는 입력

```
char line[32];
if (fgets(line, sizeof line, stdin) == NULL) return 1;
line[strcspn(line, "\n")] = '\0'; // 끝의 개행 제거

scanf("%s", buf); // 크기를 모름 → 5교시 오버플로 재현
```

31글자 넘게 입력해 보세요 — 잘리기만 하고 죽지 않는다. strcspn 은 \n 이 없으면 strlen 과 같아 안전.

오늘의 교체표

읽기	<code>fgets(buf, sizeof buf, stdin)</code>
복사	<code>snprintf(dst, sizeof dst, "%s", src)</code>
숫자 변환	<code>strtol(s, &end, 10) + end</code> 검사

⚠ strncpy 는 널을 보장하지 않는다

```
char a[4], b[4];
strncpy(a, "ABCDEFGH", 4); // A B C D — '\0' 없음!
printf("%s", a);           // 옆 메모리까지 출력

snprintf(b, sizeof b, "%s", "ABCDEFGH");
printf("%s", b);           // ABC (자르고 항상 '\0')
```

잘라서 복사할 때는 snprintf(dst, sizeof dst, "%s", src) 를 습관으로.

대신 `scanf("%s")`

대신 `strcpy` · `strncpy`

대신 `atoi` · `atof`

strtok 은 원본을 자른다 · strtol 은 끝을 알려준다

6교시

```
char line[] = "$T=25.3,H=60,D=120*"; // 배열이어야 함

char *tok = strtok(line + 1, ",");
while (tok) {
    printf("[%s] ", tok); // [T=25.3] [H=60] [D=120*]
    tok = strtok(NULL, ","); // 이어서
}
```



coma 자리에 '\0' 이 써진다 → 원본이 세 조각으로. 리터럴(char *)에 쓰면 세그폴트.

```
char *end;
long v = strtol("120*", &end, 10);
// v = 120, end → "*" (숫자가 끝난 곳)

v = strtol("abc", &end, 10);
// v = 0, end == 시작 → 실패를 알 수 있다

if (end == p) return 0; // 하나도 못 읽음
```

atoi("abc") 도 0, atoi("0") 도 0. strtol 의 end 만이 둘을 구분한다.
parse_frame 의 get_num 이 이 검사로 "T=abc" 를 BAD 로 판정.

parse_frame(line, &r): 실패하면 0 을 반환하고 out 은 건드리지 않는다 — 반쯤 채워진 구조체를 남기지 않는 것이 규칙.

📺 D2-6 strtok 분리 — 콤마가 \0 으로

힙 — 빌리고, 키우고, 돌려주기

7교시

스택으로 안 되는 두 가지

```
int *make_array(int n) {      // ① 함수가 끝나도 살아야
    // int a[n]; return a;   ← 죽은 메모리 주소
    int *a = malloc(n * sizeof(int));
    if (a == NULL) return NULL; // 항상 검사
    ...
    return a; // 호출자가 free 할 책임
}
scanf("%d", &n);              // ② 크기를 실행 중에
int *a = make_array(n);
...
free(a); a = NULL;           // 반납 + 습관
```

realloc — 줄 수를 모를 때

```
if (n == cap) {
    cap *= 2; // 두 배씩
    int *tmp = realloc(a, cap * sizeof(int));
    if (!tmp) { free(a); return 1; } // 원본은 살아 있다
    a = tmp;
}
a[n++] = v;
```

a = realloc(a, ...) 로 바로 받으면 실패 시 원본 주소를 잃어 누수.
tmp 가 관용구.
realloc 은 자리가 없으면 새 블록에 복사 후 옛 블록 해제 → 주소가 바뀔 수 있다.

소유권 규칙: 누가 free 하는지를 함수 주석에 적는다. "반환값은 호출자가 free". C 에는 GC 가 없다 — 사람이 짤을 맞춘다.

📁 D2-7 힙 할당·해제 — malloc → realloc → free

valgrind — 세 가지 실수를 눈에 익히기

7교시

누수 — free 없이 종료

```
int *p = malloc(40);
p[0] = 1;
return 0;
```

```
in use at exit: 40 bytes in 1 blocks
definitely lost: 40 bytes in 1 blocks
at malloc ... by main (leak.c:5)
```

해제 후 사용 (use-after-free)

```
free(p);
p[0] = 2;
```

```
Invalid write of size 4
Address ... is 0 bytes inside
a block of size 40 free'd
```

이중 해제 (double free)

```
free(p);
free(p);
```

```
Invalid free() / delete /
delete[] / realloc()
at free ... by main (leak.c:9)
```

```
$ gcc -Wall -Wextra -g leak.c -o leak
$ valgrind --leak-check=full ./leak
$ valgrind --leak-check=full ./analyzer < log.txt 2>&1 | grep -E "definitely|ERROR SUMMARY"
definitely lost: 0 bytes in 0 blocks / ERROR SUMMARY: 0 errors ← 제출 조건
```

MCU 에는 valgrind 가 없다. 이 세 실수는 거기서 조용히 시스템을 망가뜨린다. 오늘 PC 에서 메시지를 눈에 익혀 두는 것이 목적. free 뒤 p = NULL 습관이 ②③ 을 막는다.

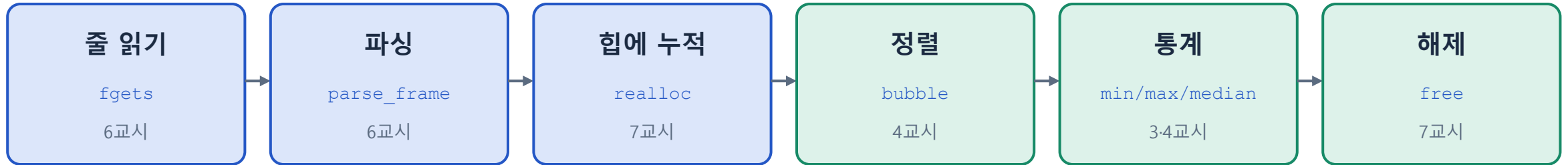
종합 실습 — 센서 로그 분석기

8교시

파일 4개 (main.c · frame.c · frame.h · Makefile) · 경고 0개 · valgrind 누수 0

줄마다 반복 (BAD 면 세고 건너뛸)

마지막에 한 번



```
$ make && ./analyzer < log.txt
BAD line 4: T=25.1,H=60,D=120*
BAD line 6: $T=abc,H=60,D=120*
min=24.8 max=37.2 median=26.0 avg=29.88 alert=2
n=5 bad=2
$ valgrind --leak-check=full ./analyzer < log.txt
definitely lost: 0 bytes in 0 blocks
```

빠대(main.c 대부분, frame.c/h 전체)는 실습 자료에 있습니다. 채울 곳은 둘: realloc 블록(tmp 관용구), 통계 계산.
조기 return 세 곳 모두에서 free(log) 를 확인하세요 — 누수는 거기서 납니다.
미완성이어도 parse_frame 통합 + realloc 루프가 있으면 통과.

제출 체크리스트

8교시



make clean && make

경고 0개 (-Wall -Wextra)



BAD 줄이 있어도 끝까지

죽지 않고 세어서 보고



입력 100줄

cap 을 여러 번 넘겨 realloc 경로가 실제로 돈다



valgrind

definitely lost 0 bytes · ERROR SUMMARY 0



worksheet_day2.html

8교시까지 채점 → 결과 코드 · 오답 정리 캡처

```
$ for i in $(seq 100); do
>   echo "\$T=$((20+i%20)).5,H=60,D=120*"
> done > big.txt
$ ./analyzer < big.txt
min=20.5 max=39.5 ... n=100 bad=0
```

제출 폴더: c_day2/analyzer/

오늘의 여섯 문장

1

& 는 주소, * 는 그 주소의 값.
원본을 바꾸려면 주소를 넘긴다

 D2-1

2

p + 1 은 sizeof(*p) 바이트 이동.
포인터 뺄셈은 요소 개수

 D2-2

3

배열은 함수에 넘기는 순간 포인터가 된다.
길이를 같이 넘겨라

 D2-3

4

배열 밖에 쓰면 컴파일러는 막지 않는다.
옆 변수가 조용히 바뀐다

 D2-4 · D2-5

5

문자열은 'W0' 까지. char[] 는 수정 가능, char* 리터럴은 읽기 전용.
fgets · snprintf · strtol 을 쓴다

 D2-5 · D2-6

6

malloc → NULL 검사 → 사용 → free.
누가 free 하는지 적어라. valgrind 로 확인

 D2-7

Day 3 예고

구조체 · 비트 연산 · 함수 포인터

오늘 만든 것들이 내일 이렇게 바뀝니다

```
sizeof(Reading) == 16 ?
```

구조체 패딩·정렬 — double 8 + int 4 + int 4

```
buf[0] | (buf[1] << 8)
```

비트 연산 — 마스킹, 시프트, 레지스터 제어

```
bubble(...) → qsort(log, n, sizeof *log, cmp)
```

함수 포인터 — 비교 함수를 넘기는 정렬

제출: c_day2/analyzer/ (파일 4개) + valgrind 캡처 + worksheet_day2.html 결과 · 내일 09:00